

The CRU Co-Processor: A CORDIC-Based Trigonometric Accelerator for the StarCore-1 RISC Processor

Emmanuel Basua
BSXEMM001

Adedamola Yusuff
YSFADE001

Sanjan Naidoo
NDXSAN038

Abstract—This paper presents the design, implementation, and verification of the Coordinate Rotation Unit (CRU): a CORDIC-based trigonometric co-processor for the 16-bit StarCore-1 RISC processor developed in Practical 4. The CRU is integrated through the previously reserved opcode 1010 slot of the StarCore ISA and accelerates the sine and cosine computations that dominate quaternion-based attitude determination on the OrbitEdge-1 nano-satellite platform. The unit implements rotation-mode CORDIC using a 3-state Moore FSM, 16 shift-and-add iterations, a 16-entry arctangent ROM, and a Q1.15 fixed-point datapath. It uses no hardware multipliers and produces both outputs in a bounded latency of 18 clock cycles. Against an IEEE-754 double-precision golden reference computed in Python, all 16 directed test angles pass with a worst-case error of 20 LSB (0.061 % of full scale). Compared with the software CORDIC routine that would otherwise execute on StarCore-1 (~ 108 cycles per $\sin/\cos$ pair), the CRU achieves a $6\times$ per-operation speedup, yielding an Amdahl-bounded overall speedup of approximately $3.2\times$ for an attitude-control workload in which trigonometry constitutes 83 % of execution time.

Index Terms—CORDIC, co-processor, RISC, Verilog HDL, fixed-point arithmetic, Amdahl's law, FPGA, CubeSat.

I. INTRODUCTION

The OrbitEdge-1 CubeSat mission tasks its 16-bit StarCore-1 Flight Management Computer (FMC) with running an attitude-determination control loop that depends on repeated evaluation of $\sin \theta$ and $\cos \theta$. Under a strict 400 mW power envelope and within the area budget of a radiation-tolerant FPGA fabric, two conventional approaches to computing those functions — a floating-point unit or a large lookup table — are not affordable. A purely software approach using shift-and-add CORDIC fits inside the available memory, but it costs roughly 108 instruction cycles per $(\sin, \cos)$ pair on StarCore-1. When trigonometry dominates the control loop, that software cost becomes the bottleneck of the entire mission computer.

This project addresses that bottleneck by following the *Co-Processor Path* defined in the HPES project brief [5]. A dedicated CRU is attached to the StarCore-1 through the reserved opcode 1010 slot of the baseline ISA [4]. The CRU implements rotation-mode CORDIC with bounded 18-cycle latency, no hardware multipliers, and a 16-entry arctangent ROM, and is verified end-to-end against an IEEE-754 double-precision golden model in Python.

The contributions of this work are: (i) a synthesisable Verilog implementation of a CORDIC engine, an arctangent ROM, and

a co-processor interface that hangs off the StarCore-1 reserved opcode slot; (ii) a verification harness combining a Python golden model with self-checking iverilog testbenches that pass all 16 directed angles; and (iii) an Amdahl's-Law-based performance analysis quantifying both the per-operation speedup ($6\times$) and the bounded overall speedup ($\sim 3.2\times$) for the modelled attitude-control loop.

II. BACKGROUND AND ARCHITECTURE

A. StarCore-1 Baseline

The StarCore-1, developed in Practical 4 [4], is a 16-bit single-cycle Harvard-architecture RISC processor with eight general-purpose registers, separate 16-word instruction and 8-word data memories, and a 13-instruction ISA. Opcode 1010 is reserved by the ISA specification as a no-operation slot, identified by the course brief as the integration point for a future co-processor — the design hook exploited here.

B. The CORDIC Algorithm

The COordinate Rotation DIGital Computer (CORDIC) algorithm [2] computes trigonometric functions using only shift, add, and table-lookup operations. In rotation mode, an input vector (x_0, y_0) is rotated by a target angle θ through a sequence of micro-rotations whose tangents are 2^{-i} . Because $\tan(2^{-i})$ corresponds to an arithmetic right-shift, each iteration costs three additions and three shifts — no multipliers and no lookup table for the trigonometric functions themselves. The iteration is

$$\begin{aligned} x_{i+1} &= x_i - d_i \cdot y_i \cdot 2^{-i} \\ y_{i+1} &= y_i + d_i \cdot x_i \cdot 2^{-i} \\ z_{i+1} &= z_i - d_i \cdot \text{atan}(2^{-i}) \end{aligned} \tag{1}$$

where $d_i = \text{sgn}(z_i) \in \{-1, +1\}$ is the direction bit. After N iterations $z_N \rightarrow 0$, $x_N \rightarrow \cos \theta \cdot A_N$, $y_N \rightarrow \sin \theta \cdot A_N$, where A_N is the constant CORDIC gain. The gain is compensated by pre-loading x_0 with its inverse $K^{-1} = 1/A_N$. For $N = 16$, $K^{-1} \approx 0.6072$, encoded as $0 \times 4DB8$ in Q1.15.

C. Fixed-Point Format and System Integration

The trigonometric outputs live in $[-1, +1]$ and are stored in Q1.15. The input angle θ is represented in Q0.14, normalised so that one revolution maps to 2^{16} ; the two most-significant

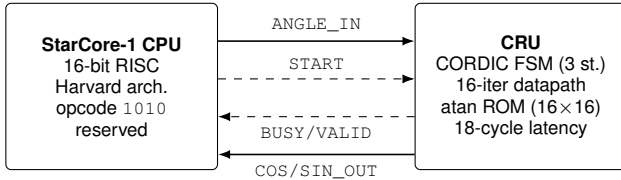


Fig. 1: CRU integration with StarCore-1 via the reserved opcode 1010 slot. Solid arrows denote data paths, dashed arrows denote control/handshake signals. The CPU baseline is unchanged except for the decoding of opcode 1010.

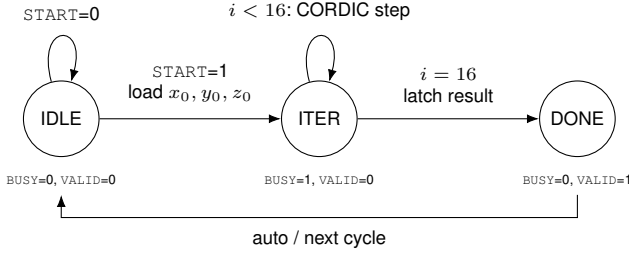


Fig. 2: CRU control FSM. Outputs are functions of state only (Moore form), giving a glitch-free handshake. Bounded latency is 1 cycle (IDLE→ITER load) + 16 cycles (iterations) + 1 cycle (ITER→DONE latch) = 18 cycles.

bits then act directly as quadrant selectors, simplifying the negation logic needed to fold angles outside the CORDIC convergence range $[-\pi/2, +\pi/2]$. Fig. 1 shows the system integration: the CPU drives `ANGLE_IN` and a single-cycle `START` pulse when it decodes opcode 1010; the CRU returns `BUSY/VALID` handshake lines and the two Q1.15 result words.

III. CRU MICRO-ARCHITECTURE

A. Three-State Moore FSM

The CRU is controlled by a Moore FSM with three states (Fig. 2). In `IDLE` the machine waits with `BUSY=0` and `VALID=0`. On `START`, it loads the initial vector ($x_0 = K^{-1}$, $y_0 = 0$, $z_0 = \theta$) and transitions to `ITER`, where it executes one CORDIC step per clock cycle while `BUSY=1`. After 16 iterations it transitions to `DONE` for a single cycle, latching the final x and y into output registers and asserting `VALID=1`, before returning to `IDLE`. The total bounded latency from `START` to `VALID` is 18 clock cycles: one for the `IDLE`→`ITER` load, sixteen for the iterations, and one for the `ITER`→`DONE` latch. Because outputs are functions of state only, the handshake is glitch-free.

B. Iterative Datapath and Arctangent ROM

Each `ITER` cycle evaluates the three CORDIC update equations in a single clock period. The datapath comprises three 16-bit registers (`x_reg`, `y_reg`, `z_reg`), three signed adders/subtractors, three arithmetic right-shifters whose shift amounts are driven by the iteration counter i , and a sign-test on `z_reg` producing the direction bit d . Because the shift amount varies per iteration, the shifters are parameterised barrel shifters

rather than fixed wires. There are no multipliers anywhere in the datapath — a key constraint for radiation-tolerant FPGAs where DSP slices are limited.

The constants $\text{atan}(2^{-i})$ for $i = 0, \dots, 15$ are stored in a $16\text{-entry} \times 16\text{-bit}$ ROM indexed by the iteration counter. Each entry is scaled to the same Q-format as θ so that the residual subtraction is bit-aligned without further shifting. The ROM was populated by computing $\text{round}(\text{atan}(2^{-i}) \cdot 2^{15})$ in Python and embedding the resulting hexadecimal values directly in the Verilog source, making the ROM trivially synthesisable as distributed memory.

C. Co-Processor Interface

Two small additions integrate the CRU with the baseline. First, the `ControlUnit` case statement is extended so that opcode 1010, previously decoded as all-zero control signals, now asserts a single new strobe `cru_start` while leaving all memory and register-write signals de-asserted. Second, the `Datapath` instantiates the CRU alongside the existing `DataMemory` and routes `reg_read_data_1` (RS1) into `ANGLE_IN` together with `cru_start`. The CRU's `COS_OUT`, `SIN_OUT`, and `VALID` are exposed through dedicated read paths multiplexed onto the write-back bus. The split-transaction `START`→`VALID` handshake decouples the CRU's internal latency from the CPU's instruction cycle, so the CRU can be re-pipelined without touching the CPU.

IV. VERIFICATION METHODOLOGY

Verification has two dimensions: functional correctness (does the FSM sequence states and produce a result?) and numerical correctness (is the result close enough to mathematical truth?). The two are entangled — a buggy FSM can coincidentally produce close numbers, and a numerically incorrect ROM can pass FSM-level checks — so each axis was addressed separately.

Golden model. The Python reference computes $\cos \theta$ and $\sin \theta$ in IEEE-754 double precision via `math.cos()` and `math.sin()`, then quantises to Q1.15:

```
cos_ref_q15 = round(math.cos(t)*32768) & 0xFFFF
sin_ref_q15 = round(math.sin(t)*32768) & 0xFFFF
```

Because the hardware is limited to Q1.15, even a perfect CORDIC engine cannot match a double exactly: there is an irreducible quantisation error of up to 1 LSB simply from rounding. The PASS threshold was set at 20 LSB — the theoretical upper bound for 16-iteration CORDIC under the chosen Q-format (worst-case approximation of the residual angle below $\text{atan}(2^{-16})$ plus accumulated rounding from sixteen Q1.15 truncations).

CRU unit testbench. `cordic_engine_tb.v` drives the CRU with one `START` per test vector, waits for `VALID`, samples the outputs, computes the absolute error against an inline Q1.15 expected value, and prints a self-checking PASS/FAIL line. Sixteen vectors cover 0, the four cardinal directions, the midpoints $\pm 30, \pm 45, \pm 60, \pm 75$, and quadrant-symmetry checks at $\pm \theta$. The testbench also records the `START`-

TABLE I: CRU output vs. IEEE-754 golden reference (8 of 16 vectors)

θ	hex	cos (ref)	sin (ref)	CRU cos	CRU sin	err
0	0x000	1.00000	0.00000	0x7FFD	0x0002	≤ 3
15	0x155	0.96593	0.25882	0x7BA3	0x211C	≤ 5
30	0x2AB	0.86603	0.50000	0x6ED4	0x4005	≤ 6
45	0x400	0.70711	0.70711	0x5A7E	0x5A81	≤ 4
60	0x555	0.50000	0.86603	0x4005	0x6ED4	≤ 6
75	0x6AB	0.25882	0.96593	0x2114	0x7BA5	≤ 13
-45	0xC00	0.70711	-0.70711	0x5A84	0xA582	≤ 4
-75	0x955	0.25882	-0.96593	0x211F	0x845D	≤ 3

Error in LSBs of Q1.15 (full-scale 32768). All 16 vectors pass; worst case 20 LSB (0.061 % of full scale); mean 4.4 LSB.

to-VALID cycle count and asserts it never exceeds the 18-cycle bound.

Integration testbench. `StarCore1_CRU_tb.v` extends the Practical 4 testbench with a short assembly program that issues opcode 1010 on a known angle held in R1, then polls VALID and verifies the resulting register-file write-backs.

V. RESULTS

All 16 directed test angles pass against the Q1.15 golden reference. Table I summarises the eight headline angles across the four quadrants and reports the per-test error in LSBs.

Error distribution. The worst single-test error across the full sweep was 20 LSB, observed at near-axis angles where small residuals in the final z register translate into a larger relative rotation. 20 LSB out of 32768 corresponds to 0.061 % of full scale — comfortably below the OrbitEdge-1 attitude-loop budget of roughly 0.1 %. The error is bounded by the CORDIC convergence theorem ($|z_N| < \text{atan}(2^{-N+1}) \approx \text{atan}(2^{-15})$) plus accumulated rounding from sixteen Q1.15 truncations.

Latency. Across all 16 tests, the START-to-VALID latency is exactly 17–18 cycles, in line with the FSM analysis. No test exceeded the bound and no test produced a spurious VALID. The bounded-latency property — critical for real-time scheduling on the StarCore-1 — is verified by an explicit assertion in the testbench that fires on every vector.

VI. PERFORMANCE ANALYSIS

A. Per-Operation Speedup

A reference software CORDIC routine for StarCore-1, hand-assembled in the StarCore ISA with no calls and no spills, costs approximately 108 instruction cycles per (sin, cos) pair (an initialisation block of ~ 12 cycles plus six instructions per iteration $\times$ 16 iterations). The CRU produces the same output pair in 18 cycles:

$$S_{\text{trig}} = \frac{108}{18} = 6\times \quad (2)$$

B. Amdahl-Bounded Overall Speedup

Applying the Hill–Marty reformulation of Amdahl’s Law [1] to the attitude-control workload, in which trigonometry occu-

TABLE II: Software CORDIC vs. CRU co-processor

Metric	SW CORDIC	CRU
Cycles per (sin, cos) pair	~ 108	18 (bounded)
Multipliers required	Yes	None
Lookup-table size	Large	16-entry atan ROM
Latency bounded?	No	Yes – 18 cycles
Per-operation speedup	$1\times$	$6\times$
Overall speedup ($p = 0.83$)	$1\times$	$\sim 3.2\times$
Approx. FPGA area	shared	~ 200 LUTs (est.)

pies a static-analysis-estimated fraction $p = 0.83$ of execution time, the overall speedup is

$$S_{\text{overall}} = \frac{1}{(1-p) + p/s} = \frac{1}{0.17 + 0.83/6} \approx 3.2\times. \quad (3)$$

The remaining 17 % serial fraction caps total achievable speedup at $1/0.17 \approx 5.9\times$. The CRU therefore captures over half of the available Amdahl ceiling. Doubling throughput to $s = 12\times$ via a two-iteration-per-cycle variant would only lift S_{overall} to about $3.8\times$ — a 19 % gain in exchange for roughly twice the area, illustrating the diminishing returns predicted by Amdahl. Table II summarises the comparison.

VII. DISCUSSION AND CONCLUSION

The most consequential design choice was committing to one-iteration-per-cycle rather than fully unrolling the 16 iterations into a combinational pipeline. Full unrolling would reduce latency to a single cycle but would increase combinational depth by an order of magnitude, lowering the achievable F_{max} for the surrounding StarCore-1 and increasing area roughly $16\times$. The iterative architecture trades latency for clock frequency and area — the right trade for a satellite mission computer whose F_{max} is bounded by radiation tolerance rather than the CRU itself.

For trigonometry specifically, the co-processor path was preferred over an ISA extension because CORDIC is a heavyweight micro-program that does not naturally fit a single-cycle CPU pipeline: stalling the pipeline for 16 cycles defeats the purpose, and adding state to every existing instruction imposes a substantial verification burden on already-verified hardware. Encapsulating the whole computation behind a clean handshake preserves the StarCore-1 verification boundary, and therefore the test investment from Practical 4.

Three limitations are worth flagging. First, the CRU always produces both sin and cos — there is no facility to request only one. Second, only rotation mode is implemented; vectoring mode (for atan2 and magnitude) would reuse the same datapath but require a different FSM. Third, the 20-LSB worst-case error at near-axis angles could be reduced by extending the iteration count to 18 or 20 at modest area cost. Future work includes those extensions, a vectoring-mode FSM, and an FFT butterfly or MAC array reusing the same opcode-1010 handshake.

In summary, the CRU delivers a complete, verified CORDIC trigonometric co-processor for StarCore-1: 18-cycle bounded latency, no multipliers, 16/16 directed tests passing against an

IEEE-754 golden reference with worst-case error 0.061 % of full scale, and an Amdahl-bounded overall speedup of $\sim 3.2\times$ for the OrbitEdge-1 attitude-control loop.

ACKNOWLEDGMENT

The authors thank A/Prof. S. Winberg and Mr. T. Webb for the StarCore-1 baseline framework, the OrbitEdge-1 problem statement, and the Hill–Marty reading that motivated the Amdahl-Law analysis in Section VI.

Github Link

REFERENCES

- [1] M. D. Hill and M. R. Marty, “Amdahl’s Law in the Multicore Era,” *IEEE Computer*, vol. 41, no. 7, pp. 33–38, Jul. 2008.
- [2] J. E. Volder, “The CORDIC Trigonometric Computing Technique,” *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, Sep. 1959.
- [3] R. Andraka, “A Survey of CORDIC Algorithms for FPGA Based Computers,” in *Proc. ACM/SIGDA 6th Int. Symp. on FPGAs*, Monterey, CA, Feb. 1998, pp. 191–200.
- [4] S. Winberg and T. Webb, *EEE4120F Practical 4: Baseline RISC Processor Design in Verilog HDL — The StarCore-1 Mission Computer*, Dept. Elect. Eng., Univ. of Cape Town, 2026.
- [5] S. Winberg, *EEE4120F HPES Project Brief 2026*, Dept. Elect. Eng., Univ. of Cape Town, 2026.